# Lab 5

## Application Load Balancer

### [1] Create 2 EC2 instances

### [2] Create an Application Load Balancer

I created 2 EC2 instances in two different availability zones in my allocated region(us-east-1) using the Python script below. I also created a security group to authorise inbound traffic for HTTP and SSH before creating the instances.

**Python script** `create_alb.py`

```
import boto3
from botocore.exceptions import ClientError

# Create a Boto3 EC2 and ELBv2 client
ec2 = boto3.client('ec2')
elbv2 = boto3.client('elbv2')

# Step 1: Create a security group
def create_security_group(group_name, description):
    try:
        response = ec2.create_security_group(
            GroupName=group_name,
            Description=description,
        )
        security_group_id = response['GroupId']
        print(f'Security Group {group_name} created with ID: {security_group_id}')
        return security_group_id
    except ClientError as e:
        print(f"Error creating security group: {e}")
        return None

# Step 2: Authorise inbound traffic for SSH and HTTP
def authorize_security_group_ingress(group_name):
    '''
    Adds inbound rules to the security group, allowing traffic for SSH (port 22)
and HTTP (port 80) from all IP addresses (0.0.0.0/0).
    '''
    try:
        ec2.authorize_security_group_ingress(
            GroupName= group_name,
            IpPermissions=[
                {
                    'IpProtocol': 'tcp',
                    'FromPort': 22, # SSH
                    'ToPort': 22,
```

```
                    'IpRanges': [{'CidrIp': '0.0.0.0/0'}]
                },
                {
                    'IpProtocol': 'tcp',
                    'FromPort': 80,  # HTTP
                    'ToPort': 80,
                    'IpRanges': [{'CidrIp': '0.0.0.0/0'}]
                }
            ]
        )
        print(f"SSH and HTTP rule added to Security Group: {group_name}")
    except ClientError as e:
        print(f"Error adding rules to security group: {e}")

# Step 3: Get the available zones in your region
def get_availability_zones():
    availability_zones = ec2.describe_availability_zones()['AvailabilityZones']

    if len(availability_zones) < 2:
        raise Exception('Less than 2 availability zones available in this
region.')

    return availability_zones

# Step 4: Create a key pair
def create_key_pair(key_name):
    try:
        key_pair = ec2.create_key_pair(KeyName=key_name)
        private_key = key_pair['KeyMaterial']
        with open(f"{key_name}.pem", "w") as key_file:
            key_file.write(private_key)
        print(f"Key pair {key_name} created and saved as {key_name}.pem")
    except ClientError as e:
        print(f"Error creating key pair: {e}")

# Step 5: Create two instances in different availability zones
def create_instances(key_name, security_group_name, ami_id, availability_zones,
student_number):
    instance_ids = []
    for i in range(2):
        try:
            instances = ec2.run_instances(
                ImageId= ami_id,
                MinCount=1,
                MaxCount=1,
                InstanceType='t2.micro',
                KeyName=key_name,
                SecurityGroups=[security_group_name],
                Placement={
                    'AvailabilityZone': availability_zones[i]['ZoneName']
                },
                TagSpecifications=[
                    {
                        'ResourceType': 'instance',
```

```
                        'Tags': [
                            {'Key': 'Name', 'Value': f'{student_number}-vm{i+1}'}
                        ]
                    }
                ]
            )
            instance_id = instances['Instances'][0]['InstanceId']
            instance_ids.append(instance_id)
            print(f"EC2 instance created with Instance ID: {instance_id}")
        except ClientError as e:
            print(f"Error creating EC2 instance: {e}")
    return instance_ids

# Step 6: Get the available subnets in your region
def get_subnets():
    subnets_response = ec2.describe_subnets()
    subnets = subnets_response['Subnets']

    # Filter subnets to only include those that are in available state
    available_subnets = [subnet for subnet in subnets if subnet['State'] ==
'available']

    if len(available_subnets) < 2:
        raise Exception('Less than 2 available subnets in this region.')

    return available_subnets

# Step 7: Create the network load balancer
def create_load_balancer(sg_id, subnets):
    try:
        response = elbv2.create_load_balancer(
            Name='22756463-load-balancer',
            Subnets=subnets,
            SecurityGroups=[sg_id],
            Scheme='internet-facing',
            Type='application',
            IpAddressType='ipv4'
        )
        lb_arn = response['LoadBalancers'][0]['LoadBalancerArn']
        print(f"Load Balancer created with ARN: {lb_arn}")
        return lb_arn
    except ClientError as e:
        print(f"Error creating load balancer: {e}")
        return None

# Step 8: Create a target group
def create_target_group(vpc_id):
    try:
        response = elbv2.create_target_group(
            Name='22756463-target-group',
            Protocol='HTTP',
            Port=80,
            VpcId=vpc_id,
            HealthCheckProtocol='HTTP',
```

```
            HealthCheckPort='80',
            TargetType='instance'
        )
        target_group_arn = response['TargetGroups'][0]['TargetGroupArn']
        print(f"Target group created with ARN: {target_group_arn}")
        return target_group_arn
    except ClientError as e:
        print(f"Error creating target group: {e}")
        return None

# Step 9: Register targets (EC2 instances) to the target group
def register_targets(target_group_arn, instance_ids):
    try:
        targets = [{'Id': instance_id} for instance_id in instance_ids]
        elbv2.register_targets(
            TargetGroupArn=target_group_arn,
            Targets=targets
        )
        print(f"Registered instances {instance_ids} to the target group.")
    except ClientError as e:
        print(f"Error registering targets: {e}")

# Step 10: Create a listener to forward traffic to the target group
def create_listener(lb_arn, target_group_arn):
    try:
        elbv2.create_listener(
            LoadBalancerArn=lb_arn,
            Protocol='HTTP',
            Port=80,
            DefaultActions=[{
                'Type': 'forward',
                'TargetGroupArn': target_group_arn
            }]
        )
        print(f"Listener created for Load Balancer {lb_arn}")
    except ClientError as e:
        print(f"Error creating listener: {e}")

def main():
    sg_name = "22756463-sg"
    sg_description = "security group for development environment"
    student_number = '22756463'
    ami_id = 'ami-0a0e5d9c7acc336f1'
    key_name = "22756463-key"

    # Step 1: Create a security group
    sg_id = create_security_group(sg_name, sg_description)

    # Step 2: Authorise inbound traffic for SSH and HTTP
    authorize_security_group_ingress(sg_name)

    # Step 3: Get the available zones in your region
    availability_zones = get_availability_zones()
```

```
    # Step 4: Create a key pair
    create_key_pair(key_name)

    # Step 5: Create two instances in different availability zones
    instance_ids = create_instances(key_name, sg_name, ami_id, availability_zones,
student_number)

    # Step 6: Get VPC ID
    vpcs = ec2.describe_vpcs()
    vpc_id = vpcs['Vpcs'][0]['VpcId']

    # Step 7: Create an Application Load Balancer (ALB)
    subnets = get_subnets()
    subnet_ids = [subnets[0]['SubnetId'], subnets[1]['SubnetId']]
    lb_arn = create_load_balancer(sg_id, subnet_ids)

    # Step 8: Create a target group in the same VPC
    target_group_arn = create_target_group(vpc_id)

    # Step 9: Register EC2 instances as targets
    register_targets(target_group_arn, instance_ids)

    # Step 10: Create a listener to forward HTTP requests to targets
    create_listener(lb_arn, target_group_arn)

if __name__ == "__main__":
    main()
```

- `elbv2.create_load_balancer()`: Creates a new Network Load Balancer, which distributes traffic across multiple targets (EC2 instances and IP addresses).
- `elbv2.create_target_group()`: Creates a target group, which is a logical grouping of EC2 instances or IP addresses to which the load balancer directs traffic.
- `ec2.create_security_group()`: Creates a security group that acts as a virtual firewall for the instances, controlling inbound and outbound traffic.
- `ec2.authorize_security_group_ingress()`: Adds inbound rules to the security group, allowing traffic for SSH (port 22) and HTTP (port 80) from all IP addresses (0.0.0.0/0).
- `ec2.describe_availability_zones()`: Retrieves a list of available availability zones in the region, ensuring that instances are created in different zones for fault tolerance.
- `ec2.create_key_pair()`: Creates a key pair for SSH access to the EC2 instances. The private key is saved locally, and the public key is associated with the instances.
- `ec2.run_instances()`: Launches EC2 instances in different availability zones, assigning them to the previously created security group and key pair.
- `elbv2.register_targets()`: Registers the EC2 instances to the target group so that the load balancer can send traffic to them.
- `elbv2.create_listener()`: Creates a listener for the load balancer that listens for HTTP traffic on port 80 and forwards the requests to the target group.

**Results**

Created a security group, authorised inbound traffic (SSH and HTTP), retrieved availability zones, created a key pair for SSH access, launched two EC2 instances with ID `i-0d275177fc9f9ac28` and `i-04cdb45522154b089`, retrieved available subnets, created an ALB with the retrieved subnets, created a target group, registered the EC2 instances with the target group, and set up a listener on the ALB to forward HTTP traffic.

## [3] Test the Application Load Balancer

### 3.1 SSH to each of the two instances

I obtained the public IP address of each EC2 instances from the AWS console. They are `98.81.236.110` and `34.204.0.107`. Then I ssh to each of the two instances using commands `ssh -i 22756463-key.pem ubuntu@98.81.236.110` and `ssh -i 22756463-key.pem ubuntu@34.204.0.107`.

#### Commands

- `ssh`: This is the command to initiate an SSH (Secure Shell) connection, which allows you to remotely connect to another machine securely over the internet.
- `-i`: The -i option specifies the identity file (in this case, 22756463-key.pem). This is the private key file that corresponds to the public key stored on my EC2 instance.
- `ubuntu`: This is the username for my EC2 instance. Since my instance is running Ubuntu, the default username is ubuntu.

### 3.2 Update each instance

I updated each instance using command `sudo apt-get update` to prepare the system to install or upgrade packages based on the latest information from the repositories.

#### Commands

- `sudo`: Run the command with elevated (root) permissions. Installing software or making changes to system-wide configurations requires superuser privileges, so sudo is necessary for this command.
- `apt-get`: managing packages on Linux. It performs low-level operations, including installing, upgrading, and removing software packages, as well as updating the local package index.
- `update`: update the package index. It retrieves the latest list of available packages, versions, and their dependencies from the package repositories configured in the system. `sudo apt-get update` updates the local list of available software packages and their versions from the repositories but does not install or upgrade any software.

### 3.3 Install apache2 in each instance

I installed apache2 by using command `sudo apt install apache2`.

#### Commands

- `sudo`: Run the command with elevated (root) permissions. Installing software or making changes to system-wide configurations requires superuser privileges, so sudo is necessary for this command.
- `apt`: The Advanced Package Tool, which is the package manager for Ubuntu. It retrieves the software packages from the distribution's software repositories.

- `install`: The operation to install the specified package.
- `apache2`: This is the name of the package I want to install. Apache 2 is a popular *web server software*. Apache 2 is used to *host websites and serve webpages over the HTTP/HTTPS protocols*.

### 3.4 Edit HTML tags

I edited the `<title>` and `</title>` tags by using command `sudo nano /var/www/html/index.html`.

#### Commands

- `sudo`: Run the command with elevated (root) permissions.
- `nano`: Open, edit and save text tiles directly from the terminal.
- `/var/www/html/index.html`: The file path to the `index.html` file I want to edit. `/var/www/html/` is the *default directory where Apache web server stores website files*. The `index.html` file is typically the default web page served when someone accesses the website hosted on the server.

### 3.5 Access instances by IP addresses

I accessed each instance by their IP addresses in the Chrome browser. Their instance names were shown as the webpage title in the top left corner. I also inspected the `<title>` tags to see the instance names.